

PRESSURE DETECTION

Mastering Embedded System Diploma

Design & Implementation Report

Author

Eng. Adel Shata

Embedded Systems
Software Engineering

GitHub: github.com/Adel-Shata

LinkedIn: [linkedin.com/in/adel-shata-a1b9a7376](https://www.linkedin.com/in/adel-shata-a1b9a7376)

Email: adel.shata.eng@gmail.com

August 2026

1. Overview

A bare-metal embedded C project targeting an STM32F103C6 (ARM Cortex-M3) microcontroller, simulated in Proteus VSM. The system monitors cabin pressure and triggers an alarm when a configurable threshold is exceeded, following SysML/UML modelling done in TTool.

The software is decomposed into four independent blocks, each implemented as a separate .c/.h module with file-scope (static) state and explicit interface functions — mirroring the AVATAR/TTool block diagram signals. The project includes a custom linker script and startup file written from scratch (no HAL, no CMSIS).

2. Case Study

Scenario: Monitor aircraft cabin pressure and alert the crew when pressure exceeds safe limits.

PARAMETER	VALUE
Target MCU	STM32F103C6 (ARM Cortex-M3)
Pressure threshold	20 bar
Alarm duration	60 seconds
Sensor input	GPIOA PA0–PA7 (8-bit DIP-switch bank)
Alarm output	PA13 (active-low LED)

3. System Architecture

The system is decomposed into four independent blocks, each as a separate .c/.h module:

MODULE	RESPONSIBILITY	INTERFACE FUNCTIONS
Pressure_Sensor_Driver	Reads raw pressure from hardware (GPIO)	pressureSensorInit(), pressureSensorRead()
High_Pressure_Detection	Compares reading against threshold, raises flag	highPressureDetectionfsmTick(), highPressureDetected()
Alarm	Reacts to high-pressure flag, drives alarm for 60s	alarmfsmTick()
Alarm_Actuator_Driver	Reactive actuator driver; turns alarm on/off	alarmActuatorInit(), alarmActuatorOn(), alarmActuatorOff()

Communication model:

All blocks communicate through explicit interface functions declared in their headers. There are no shared global variables between blocks — each block's internal state is kept static (file-scope only). The system runs a single super-loop that calls each block's tick function synchronously.

→ Block Diagram (TTool)

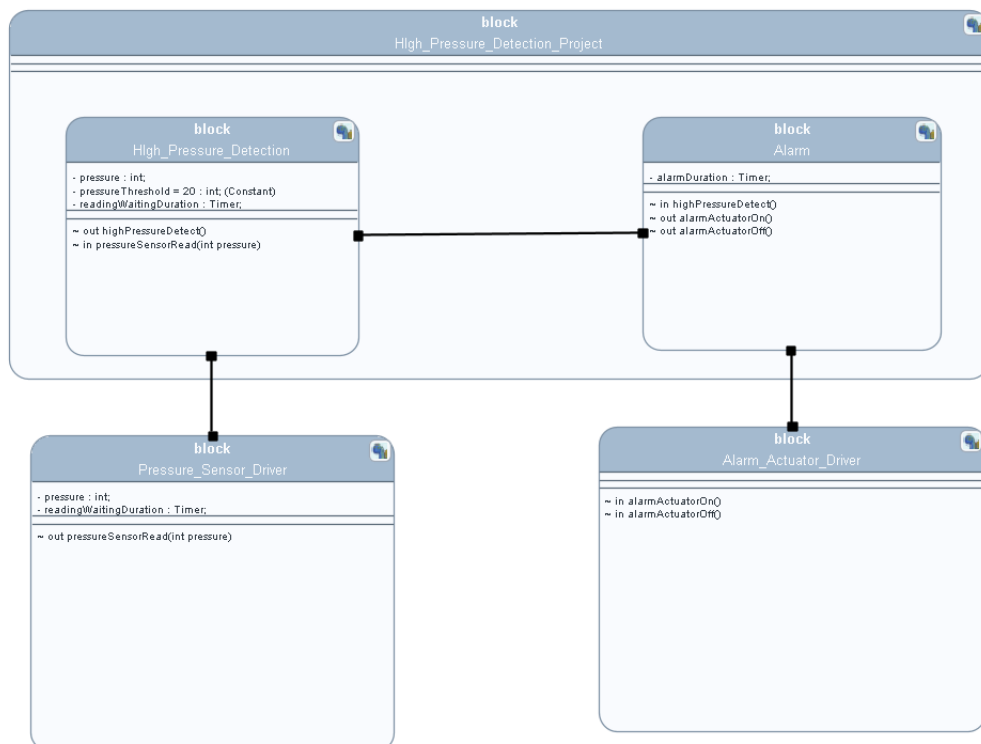


Figure 1 — System Block Diagram

4. Design Documentation (TTool / SysML / UML)

Full SysML/UML diagrams were produced in TTool and are included below ↓↓.

→ Requirement Diagram

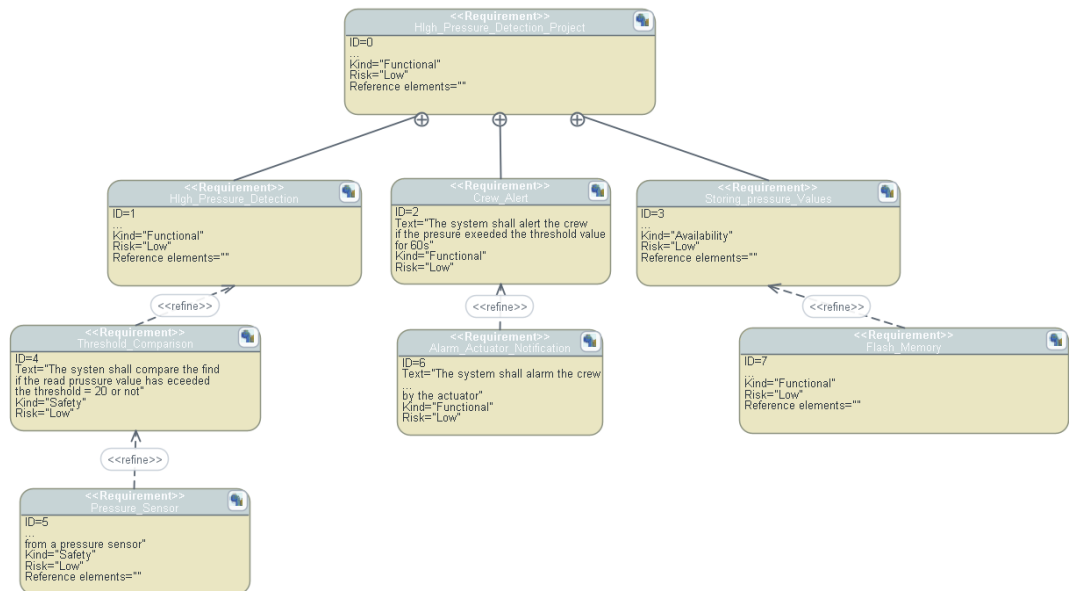


Figure 2 — Requirement Diagram

→ Use Case Diagram

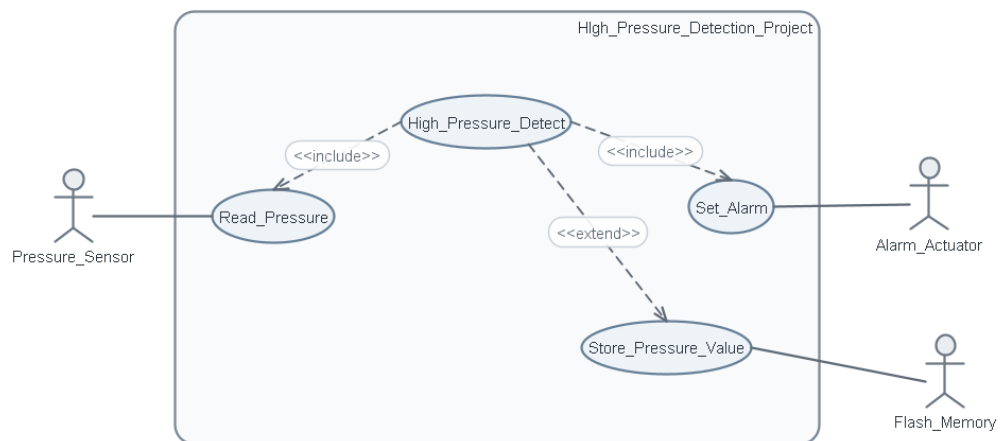


Figure 3 — Use Case Diagram

→ Activity Diagram

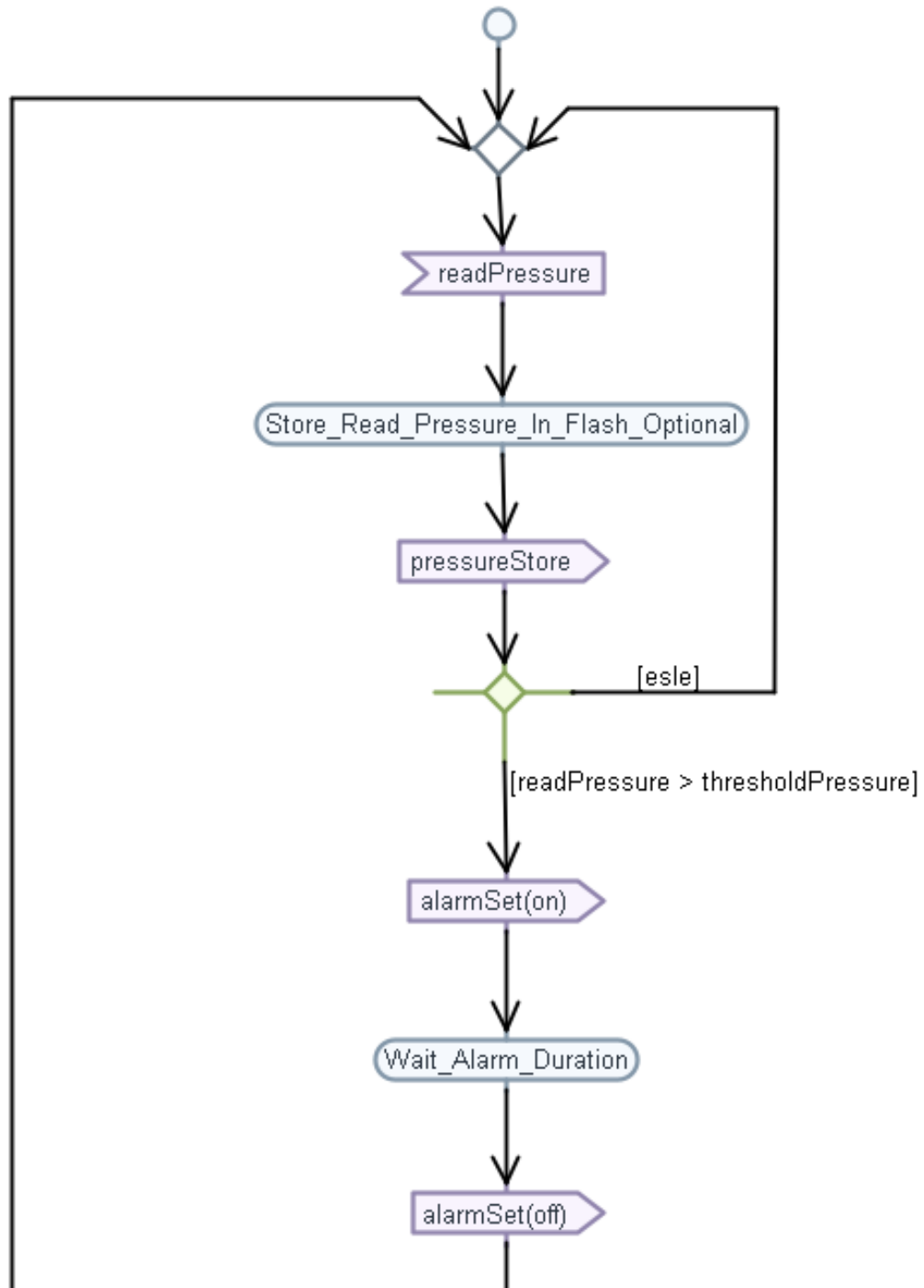


Figure 4 — Activity Diagram

→ Sequence Diagram

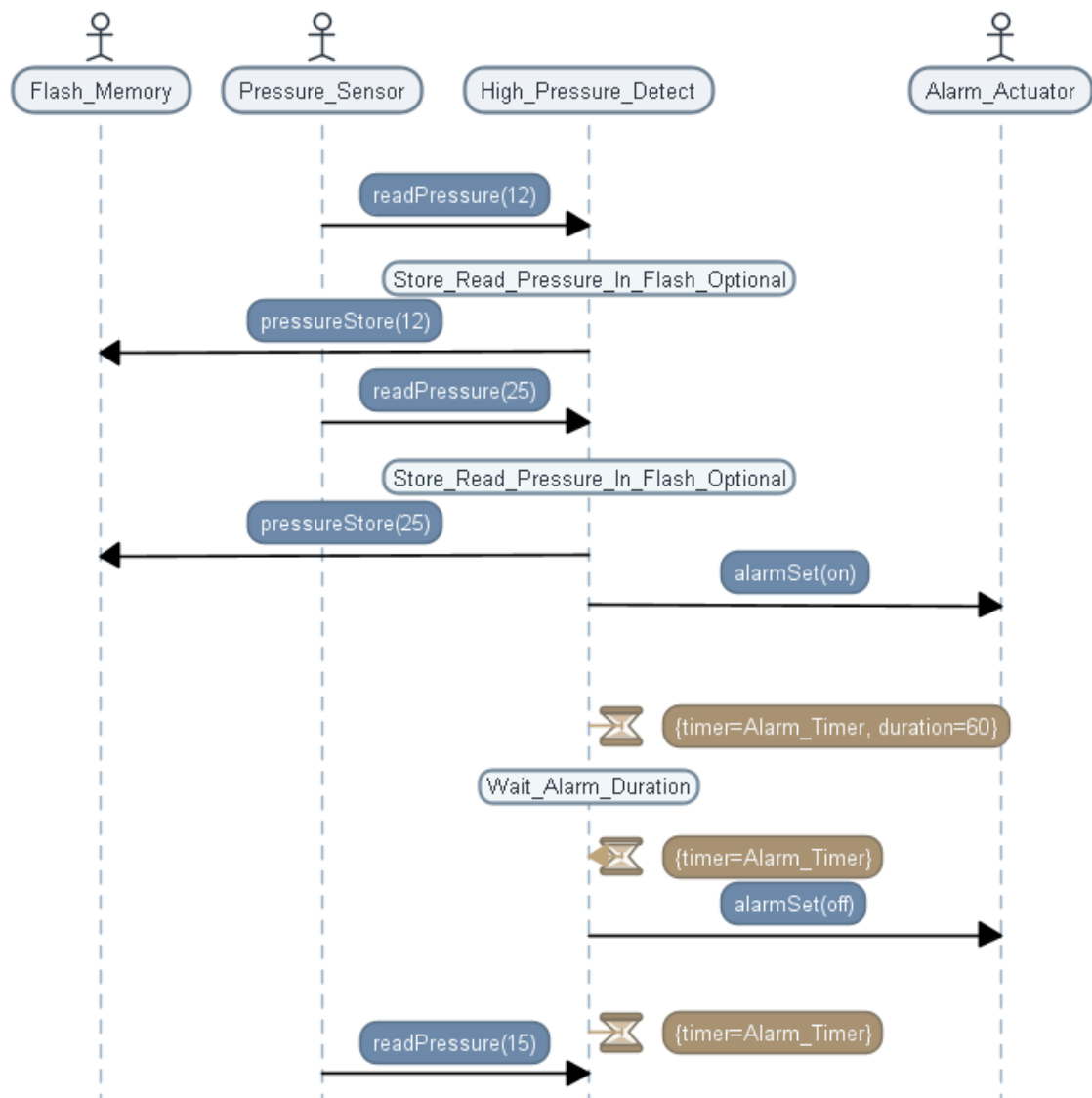


Figure 5 — Sequence Diagram

5. State Machine Diagrams

Each software block has a corresponding state machine diagram modelled in TTool.

→ Pressure Sensor Driver

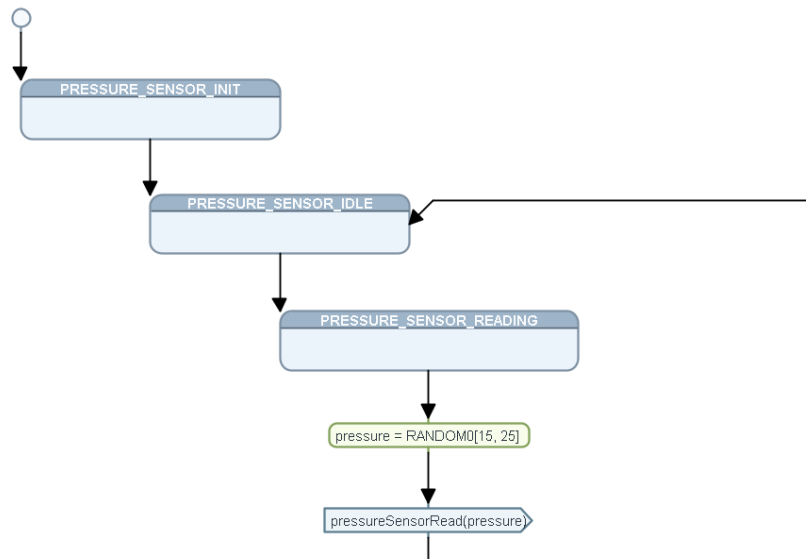


Figure 6 — State Machine — Pressure Sensor Driver

STATE	DESCRIPTION
PRESSURE_SENSOR_INIT	Initial state after pressureSensorInit()
PRESSURE_SENSOR_IDLE	Waiting for read request
PRESSURE_SENSOR_READING	Actively reading pressure value from GPIO

→ High Pressure Detection

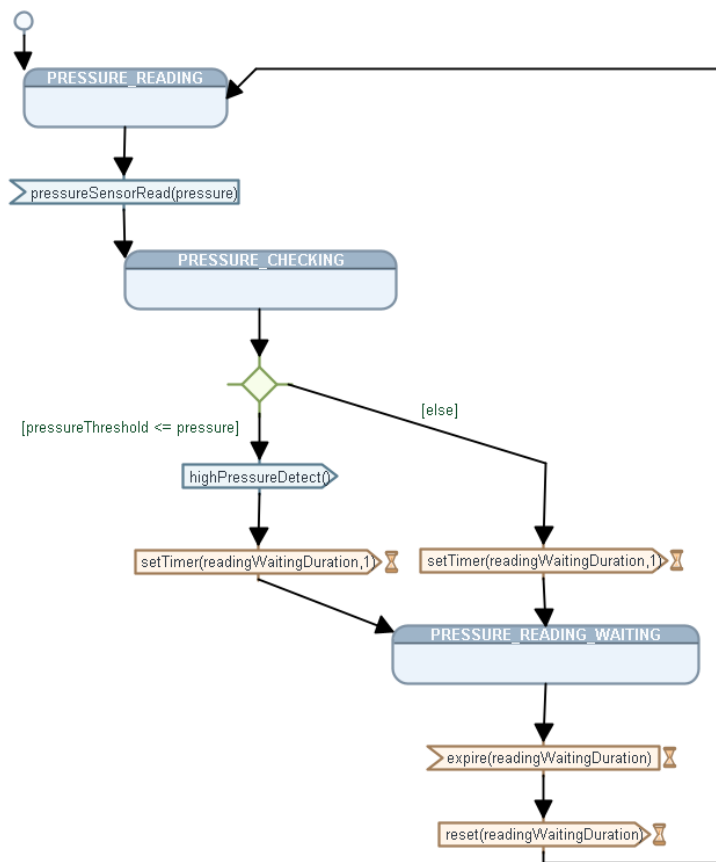


Figure 7 — State Machine — High Pressure Detection

STATE	DESCRIPTION
PRESSURE_READING	Call <code>pressureSensorRead()</code> to get current value
PRESSURE_CHECKING	Compare reading against <code>HIGH_PRESSURE_THRESHOLD (20)</code>
PRESSURE_READING_WAITING	Wait 1 second before next reading cycle

→ Alarm

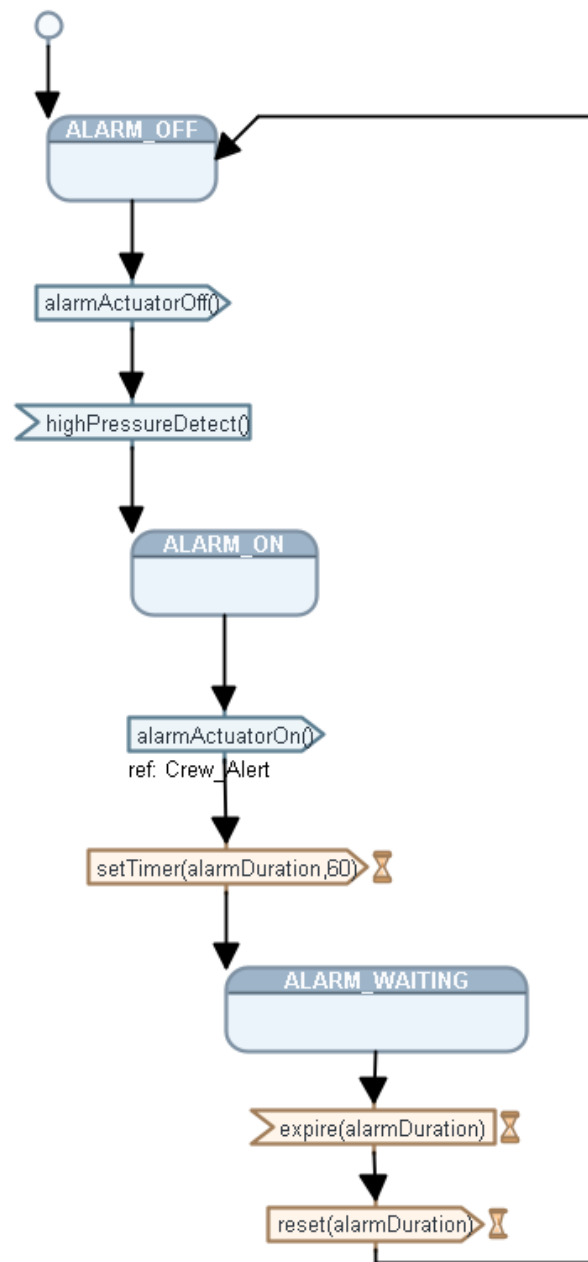


Figure 8 — State Machine — Alarm

STATE	DESCRIPTION
ALARM_OFF	Alarm inactive; check highPressureDetected()
ALARM_ON	Alarm active; call alarmActuatorOn()
ALARM_WAITING	Wait 60 seconds (delay(60000)) before turning off

→ Alarm Actuator Driver

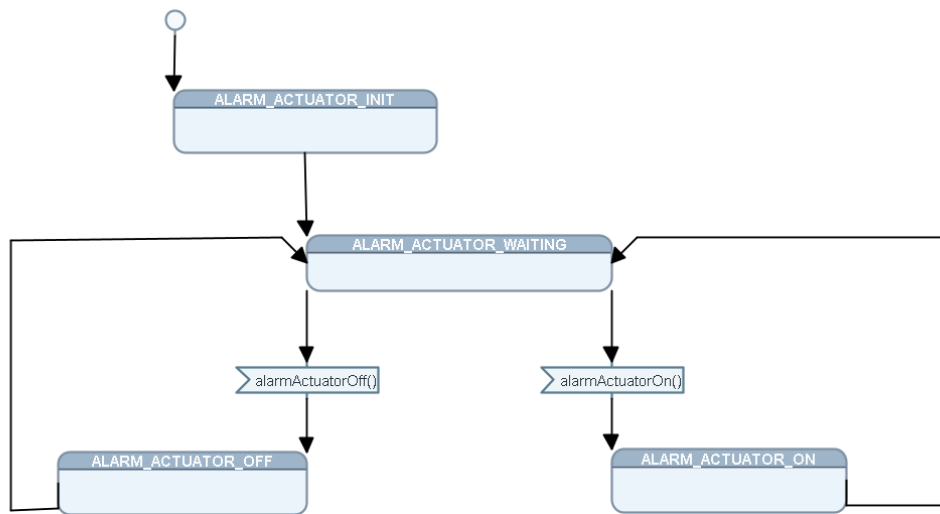


Figure 9 — State Machine — Alarm Actuator Driver

STATE	DESCRIPTION
ALARM_ACTUATOR_INIT	Initial state after alarmActuatorInit()
ALARM_ACTUATOR_WAITING	Ready to accept on/off commands
ALARM_ACTUATOR_ON	Alarm output active (PA13 LOW)
ALARM_ACTUATOR_OFF	Alarm output inactive (PA13 HIGH)

6. Source Code

The implementation follows a strict one-block-per-module structure. Every block's internal state is kept static (file-scope only), preserving encapsulation between blocks.

→ **main.c**

```
C main.c X
src > C main.c > main()
1  #include <stdint.h>
2  #include <stdio.h>
3
4
5  #include "Alarm.h"
6  #include "High_Pressure_Detection.h"
7
8  void setup(){
9      gpioInit();
10     pressureSensorInit();
11     alarmActuatorInit();
12 }
13
14 int main(){
15     setup();
16     while (1)
17     {
18         alarmfsmTick();
19         highPressureDetectionfsmTick();
20     }
21 }
22
23
```

Figure 10 — main.c

The entry point. Calls setup() to initialize all modules, then runs a super-loop calling alarmfsmTick() and highPressureDetectionfsmTick() repeatedly.

→ Low-Level Driver — driver.c / driver.h

```
C driver.c X
src > driver > C driver.c > setAlarmActuator(int)
1  #include "driver/driver.h"
2  #include <stdint.h>
3  #include <stdio.h>
4
5  void delay(int nCount)
6  {
7      for(; nCount != 0; nCount--);
8  }
9
10 int getPressureVal(){
11     return (GPIOA_IDR & 0xFF);
12 }
13
14 void setAlarmActuator(int i){
15     if (i == 0){
16         SET_BIT(GPIOA_ODR,13);
17     }
18     else if (i == 1){
19         RESET_BIT(GPIOA_ODR,13);
20     }
21 }
22
23 void gpioInit(){
24     SET_BIT(APB2ENR, 2);
25     GPIOA_CRL &= 0xFF0FFFFFFF;
26     GPIOA_CRL |= 0x00000000;
27     GPIOA_CRH &= 0xFF0FFFFFFF;
28     GPIOA_CRH |= 0x22222222;
29 }
30
```

Figure 11 — driver.c

```

C driver.h X
inc > driver > C driver.h > ...
1  #ifndef DRIVER_H_
2  #define DRIVER_H_
3
4  #include <stdint.h>
5  #include <stdio.h>
6
7  #define SET_BIT(ADDRESS,BIT)  ADDRESS |= (1<<BIT)
8  #define RESET_BIT(ADDRESS,BIT) ADDRESS &= ~(1<<BIT)
9  #define TOGGLE_BIT(ADDRESS,BIT) ADDRESS ^= (1<<BIT)
10 #define READ_BIT(ADDRESS,BIT) ((ADDRESS) & (1<<(BIT)))
11
12
13 #define GPIO_PORTA 0x40010800
14 #define BASE_RCC   0x40021000
15
16 #define APB2ENR    *(volatile uint32_t *)(BASE_RCC + 0x18)
17
18 #define GPIOA_CRL  *(volatile uint32_t *)(GPIO_PORTA + 0x00)
19 #define GPIOA_CRH  *(volatile uint32_t *)(GPIO_PORTA + 0x04)
20 #define GPIOA_IDR  *(volatile uint32_t *)(GPIO_PORTA + 0x08)
21 #define GPIOA_ODR  *(volatile uint32_t *)(GPIO_PORTA + 0x0C)
22
23
24 void delay(int nCount);
25 int  getPressureVal();
26 void setAlarmActuator(int i);
27 void gpioInit();
28
29 #endif // DRIVER_H_

```

Figure 12 — driver.h

Hardware abstraction layer. Provides:

- `gpioInit()` — configure GPIOA (enable clock, set pin modes)
- `getPressureVal()` — read 8-bit value from `GPIOA_IDR`
- `setAlarmActuator(int i)` — control PA13 (active-low)
- `delay(int nCount)` — busy-wait delay loop
- Register macros: `SET_BIT`, `RESET_BIT`, `TOGGLE_BIT`, `READ_BIT`

→ Pressure Sensor Driver — Pressure_Sensor_Driver.c / .h

```

C Pressure_Sensor_Driver.c X
src > driver > C Pressure_Sensor_Driver.c > __unnamed_enum_0417_1
1  #include "driver/Pressure_Sensor_Driver.h"
2
3  static enum{
4      PRESSURE_SENSOR_IDLE = 0,
5      PRESSURE_SENSOR_READING
6  }pressureSensorState;
7
8
9  void pressureSensorInit(){
10     pressureSensorState = PRESSURE_SENSOR_IDLE;
11 }
12
13 void pressureSensorRead(int32_t *pressure){
14     if(pressureSensorState == PRESSURE_SENSOR_IDLE){
15         pressureSensorState = PRESSURE_SENSOR_READING;
16         *pressure = getPressureVal();
17     }
18     pressureSensorState = PRESSURE_SENSOR_IDLE;
19 }

```

Figure 13 — Pressure_Sensor_Driver.c

```

C Pressure_Sensor_Driver.h X
inc > driver > C Pressure_Sensor_Driver.h > ...
1  #ifndef PRESSURE_SENSOR_DRIVER_H_
2  #define PRESSURE_SENSOR_DRIVER_H_
3
4  /*Section 1: Include Files*/
5  #include "driver.h"
6  /*Section 2: Macros*/
7  /*Section 3: Macro Functions*/
8  /*Section 4: Function Prototypes*/
9
10 /**
11  * @brief Initialize the pressure sensor
12  */
13 void pressureSensorInit();
14
15 /**
16  * @brief Read the pressure value from the sensor
17  * @param pressure Pointer to store the read pressure value
18  */
19 void pressureSensorRead(int32_t *pressure);
20 #endif // PRESSURE_SENSOR_DRIVER_H_

```

Figure 14 — Pressure_Sensor_Driver.h

FSM-based driver that reads pressure from hardware. Transitions: IDLE → READING → getPressureVal() → IDLE.

→ High Pressure Detection — High_Pressure_Detection.c / .h

```

C High_Pressure_Detection.c X
src > C High_Pressure_Detection.c > highPressureDetectionfsmTick(void)
1  #include "High_Pressure_Detection.h"
2
3  static enum{
4      PRESSURE_READING,
5      PRESSURE_CHECKING,
6      PRESSURE_READING_WAITING
7  }pressureDetectionState = PRESSURE_READING;
8
9  static highPressureState_t pressureValue = PRESSURE_NOT_HIGH;
10 static int32_t pressure;
11
12 void highPressureDetectionfsmTick(void){
13     switch(pressureDetectionState){
14         case PRESSURE_READING:
15             pressureSensorRead(&pressure);
16             pressureDetectionState = PRESSURE_CHECKING;
17             break;
18         case PRESSURE_CHECKING:
19             if(pressure >= HIGH_PRESSURE_THRESHOLD){
20                 pressureValue = PRESSURE_HIGH;
21             } else {
22                 pressureValue = PRESSURE_NOT_HIGH;
23             }
24             pressureDetectionState = PRESSURE_READING_WAITING;
25             break;
26         case PRESSURE_READING_WAITING:
27             delay(1000); // wait 1 second before next reading
28             pressureDetectionState = PRESSURE_READING;
29             break;
30     }
31 }
32
33 highPressureState_t highPressureDetected(void){
34     return pressureValue;
35 }

```

Figure 15 — High_Pressure_Detection.c

```

C High_Pressure_Detection.h X
inc > C High_Pressure_Detection.h > HIGH_PRESSURE_THRESHOLD
1  #ifndef HIGH_PRESSURE_DETECTION_H_
2  #define HIGH_PRESSURE_DETECTION_H_
3
4  /*Section 1: Include Files*/
5  #include "driver/Pressure_Sensor_Driver.h"
6  /*Section 2: User Defined Types*/
7  typedef enum{
8      PRESSURE_NOT_HIGH,
9      PRESSURE_HIGH
10 }highPressureState_t;
11 /*Section 2: Macros*/
12 #define HIGH_PRESSURE_THRESHOLD 20 // Example threshold value
13 /*Section 3: Macro Functions*/
14 /*Section 4: Function Prototypes*/
15
16 /**
17  * @brief Finite State Machine tick function for high pressure detection
18  */
19 void highPressureDetectionfsmTick(void);
20
21 /**
22  * @brief Detects if the pressure is high
23  * @return The state of the pressure detection
24  */
25 highPressureState_t highPressureDetected(void);
26
27
28 #endif // HIGH_PRESSURE_DETECTION_H_

```

Figure 16 — High_Pressure_Detection.h

FSM that reads pressure, compares against threshold (20 bar), and sets the PRESSURE_HIGH / PRESSURE_NOT_HIGH flag. Includes 1-second delay between readings.

→ Alarm — Alarm.c / .h

```

C Alarm.c X
src > C Alarm.c > alarmfsmTick(void)
1  #include "Alarm.h"
2
3  static enum{
4      ALARM_OFF,
5      ALARM_ON,
6      ALARM_WAITING
7  }alarmState = ALARM_OFF;
8
9
10 void alarmfsmTick(void) {
11     switch(alarmState){
12         case ALARM_OFF:
13             alarmActuatorOff();
14             if(highPressureDetected() == PRESSURE_HIGH){
15                 alarmState = ALARM_ON;
16             }
17             break;
18         case ALARM_ON:
19             alarmActuatorOn();
20             alarmState = ALARM_WAITING;
21             break;
22         case ALARM_WAITING:
23             delay(60000); // Wait 30 seconds before turning off the alarm
24             alarmState = ALARM_OFF;
25             break;
26     }
27 }

```

Figure 17 — Alarm.c

```

C Alarm.h X
inc > C Alarm.h > ...
1  #ifndef ALARM_H_
2  #define ALARM_H_
3  /*Section 1: Include Files*/
4  #include "driver/Alarm_Actuator_Driver.h"
5  #include "High_Pressure_Detection.h"
6  /*Section 2: Macros*/
7  /*Section 3: Macro Functions*/
8  /*Section 4: Function Prototypes*/
9
10 /**
11  * @brief Finite State Machine tick function for alarm control
12  */
13 void alarmfsmTick(void);
14
15 #endif /* ALARM_H_ */
16

```

Figure 18 — Alarm.h

FSM that monitors the high-pressure flag and controls the alarm for 60 seconds. Transitions: OFF → ON → WAITING (60s) → OFF.

→ Alarm Actuator Driver — Alarm_Actuator_Driver.c / .h

```

C Alarm_Actuator_Driver.c X
src > driver > C Alarm_Actuator_Driver.c > [E] ALARM_ACTUATOR_STATE
1  #include "driver/Alarm_Actuator_Driver.h"
2
3  static enum{
4      ALARM_ACTUATOR_OFF = 0,
5      ALARM_ACTUATOR_ON,
6      ALARM_ACTUATOR_WAITING
7  }ALARM_ACTUATOR_STATE;
8
9  void alarmActuatorInit() {
10     ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_WAITING;
11 }
12
13 void alarmActuatorOn() {
14     if(ALARM_ACTUATOR_STATE == ALARM_ACTUATOR_WAITING) {
15         ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_ON;
16         setAlarmActuator(ALARM_ACTUATOR_ON);
17         ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_WAITING;
18     }
19 }
20
21 void alarmActuatorOff() {
22     if(ALARM_ACTUATOR_STATE == ALARM_ACTUATOR_WAITING) {
23         ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_OFF;
24         setAlarmActuator(ALARM_ACTUATOR_OFF);
25         ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_WAITING;
26     }
27 }

```

Figure 19 — Alarm_Actuator_Driver.c

```

C Alarm_Actuator_Driver.h X
inc > driver > C Alarm_Actuator_Driver.h > ...
1  #ifndef ALARM_ACTUATOR_DRIVER_H_
2  #define ALARM_ACTUATOR_DRIVER_H_
3
4  /*Section 1: Include Files*/
5  #include "driver.h"
6  /*Section 2: Macros*/
7  /*Section 3: Macro Functions*/
8  /*Section 4: Function Prototypes*/
9
10 /**
11  * @brief Initialize the alarm actuator
12  */
13 void alarmActuatorInit();
14
15 /**
16  * @brief Turn the alarm actuator on
17  */
18 void alarmActuatorOn();
19
20 /**
21  * @brief Turn the alarm actuator off
22  */
23 void alarmActuatorOff();
24 #endif // ALARM_ACTUATOR_DRIVER_H_

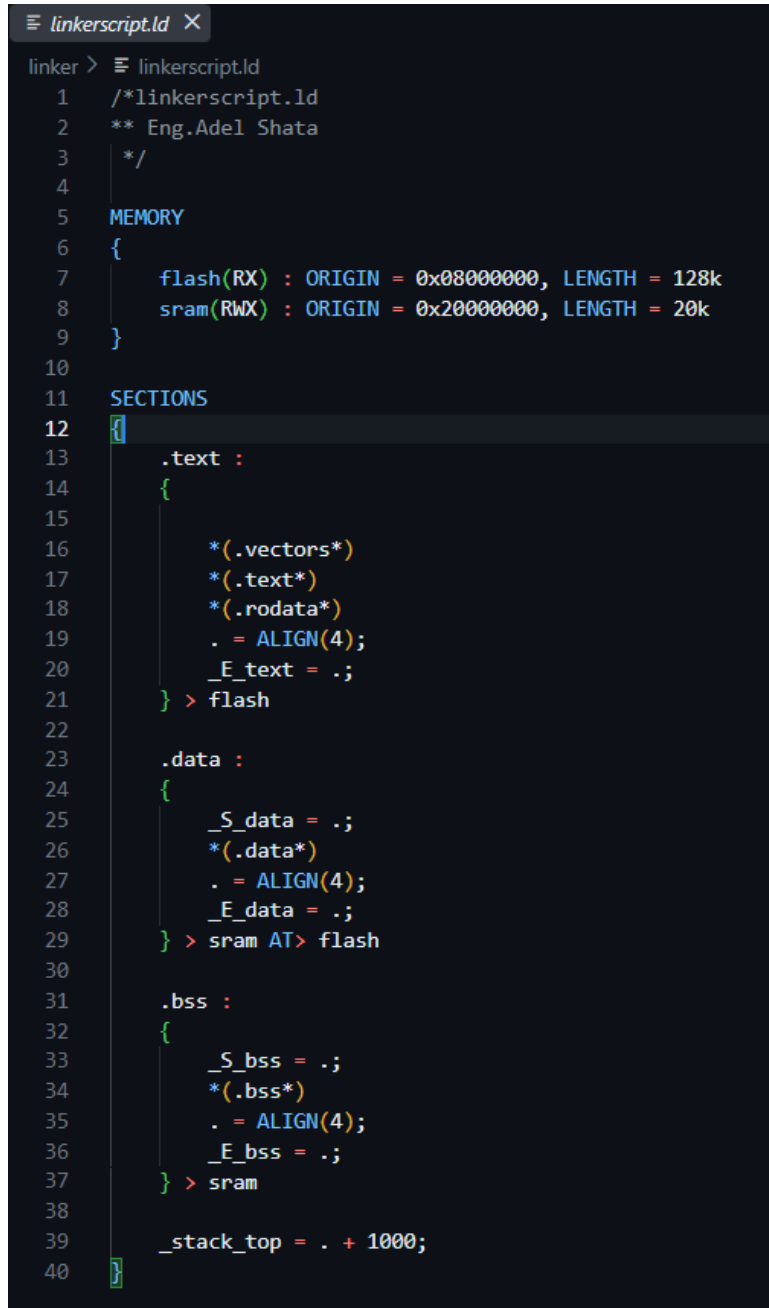
```

Figure 20 — Alarm_Actuator_Driver.h

Reactive driver that controls the physical alarm output. Only accepts commands when in WAITING state.

7. Startup & Linker

→ Linker Script — linkerscript.ld



```

linker > linkerscript.ld
1  /*linkerscript.ld
2  ** Eng.Adel Shata
3  */
4
5  MEMORY
6  {
7      flash(RX) : ORIGIN = 0x08000000, LENGTH = 128k
8      sram(RWX) : ORIGIN = 0x20000000, LENGTH = 20k
9  }
10
11  SECTIONS
12  {
13      .text :
14      {
15          *(.vectors*)
16          *(.text*)
17          *(.rodata*)
18          . = ALIGN(4);
19          _E_text = .;
20      } > flash
21
22
23      .data :
24      {
25          _S_data = .;
26          *(.data*)
27          . = ALIGN(4);
28          _E_data = .;
29      } > sram AT> flash
30
31      .bss :
32      {
33          _S_bss = .;
34          *(.bss*)
35          . = ALIGN(4);
36          _E_bss = .;
37      } > sram
38
39      _stack_top = . + 1000;
40  }

```

Figure 21 — Linker Script

Custom GNU LD linker script defining:

- FLASH: 0x08000000, 128 KB
- SRAM: 0x20000000, 20 KB
- Sections: .text (vectors + code + rodata → flash), .data (initialized → SRAM, load in flash), .bss (zero-initialized → SRAM)
- _stack_top = . + 1000

→ Startup File — startup.c (Page 1)

```

C startup.c X
startup > C startup.c > ...
1  #include "Platform_Types.h"
2
3  extern uint32 _stack_top;
4  extern uint32 _E_text;
5  extern uint32 _S_data;
6  extern uint32 _E_data;
7  extern uint32 _S_bss;
8  extern uint32 _E_bss;
9
10 extern int main(void);
11
12 #define Reserved (uint32)0
13 void Reset_Handler(void);
14 void NMI_Handler(void) __attribute__((weak, alias("Reset_Handler")));
15 void HardFault_Handler(void) __attribute__((weak, alias("Reset_Handler")));
16 void MemoryManagement_Handler(void) __attribute__((weak, alias("Reset_Handler")));
17 void BusFault_Handler(void) __attribute__((weak, alias("Reset_Handler")));
18 void UsageFault_Handler(void) __attribute__((weak, alias("Reset_Handler")));
19 void SVC_Handler(void) __attribute__((weak, alias("Reset_Handler")));
20 void PendSV_Handler(void) __attribute__((weak, alias("Reset_Handler")));
21 void SysTick_Handler(void) __attribute__((weak, alias("Reset_Handler")));
22 void ExternalInterruptsHandler(void) __attribute__((weak, alias("Reset_Handler")));
23
24 // initialize the vector table in flash memory
25 uint32 __attribute__((section(".vectors"))) vectors[] = {
26     (uint32) &_stack_top,
27     (uint32) &Reset_Handler,
28     (uint32) &NMI_Handler,
29     (uint32) &HardFault_Handler,
30     (uint32) &MemoryManagement_Handler,
31     (uint32) &BusFault_Handler,
32     (uint32) &UsageFault_Handler,
33     Reserved,
34     Reserved,
35     Reserved,
36     Reserved,
37     (uint32) &SVC_Handler,
38     Reserved,
39     Reserved,
40     (uint32) &PendSV_Handler,
41     (uint32) &SysTick_Handler,
42     (uint32) &ExternalInterruptsHandler
43 };
44

```

Figure 22 — Startup — Vector Table

Custom C-based startup file. Defines the interrupt vector table with weak aliases for all fault handlers (default: reset on any fault).

→ Startup File — startup.c (Page 2)

```
45
46 void Reset_Handler(void)
47 {
48     void *src = NULL;
49     void *dst = NULL;
50
51     // Reallocate the .data section from flash to SRAM
52     dst = (uint32 *)&_S_data;
53     src = (uint32 *)&_E_text;
54     uint32 _data_size = (uint32 *)&_E_data - (uint32 *)&_S_data;
55     for (uint32 i = 0; i < _data_size; i++){
56         *((uint32 *)dst++) = *((uint32 *)src++);
57     }
58
59     // allocate the .data section in SRAM
60     dst = (uint32 *)&_S_bss;
61     uint32 _bss_size = (uint32 *)&_E_bss - (uint32 *)&_S_bss;
62     for (uint32 i = 0; i < _bss_size; i++){
63         *((uint32 *)dst++) = Reserved;
64     }
65
66     // branch to main function
67     main();
68 }
```

Figure 23 — Startup — Reset Handler

Reset_Handler() — copies .data from FLASH to SRAM, zeros .bss, then calls main().

8. Build System

This project is built using a custom, reusable Makefile build engine maintained as a separate repository:

Universal_Make_Engine ❤️ https://github.com/Adel-Shata/Universal_Make_Engine.git

Custom Makefile-based build engine for bare-metal ARM projects.

Consult the linked repository for full documentation on configuration, usage, and available build options.

9. Simulation

The project was verified in Proteus VSM with a DIP-switch bank simulating the pressure sensor input and an LED simulating the alarm actuator output.

→ High Pressure Detected (pressure = 26, threshold = 20)

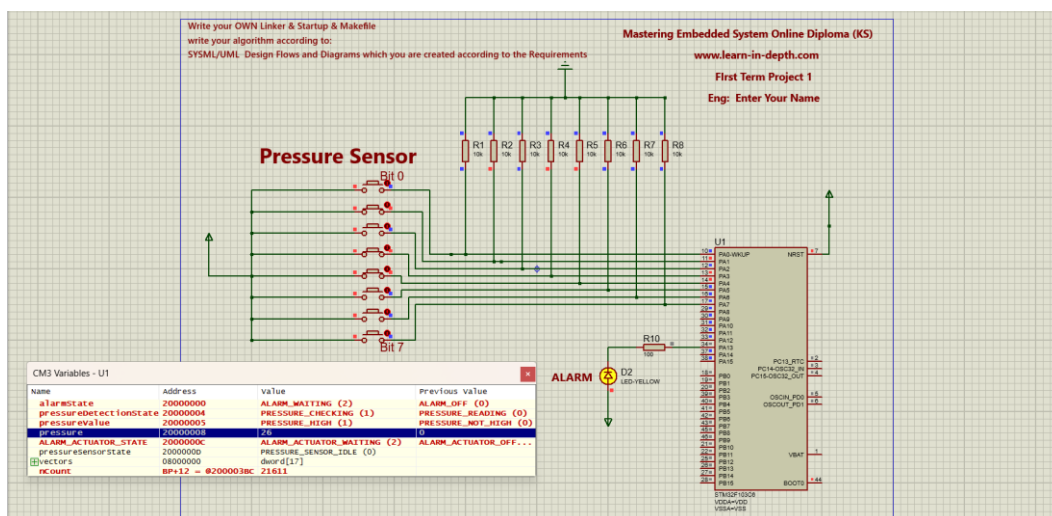
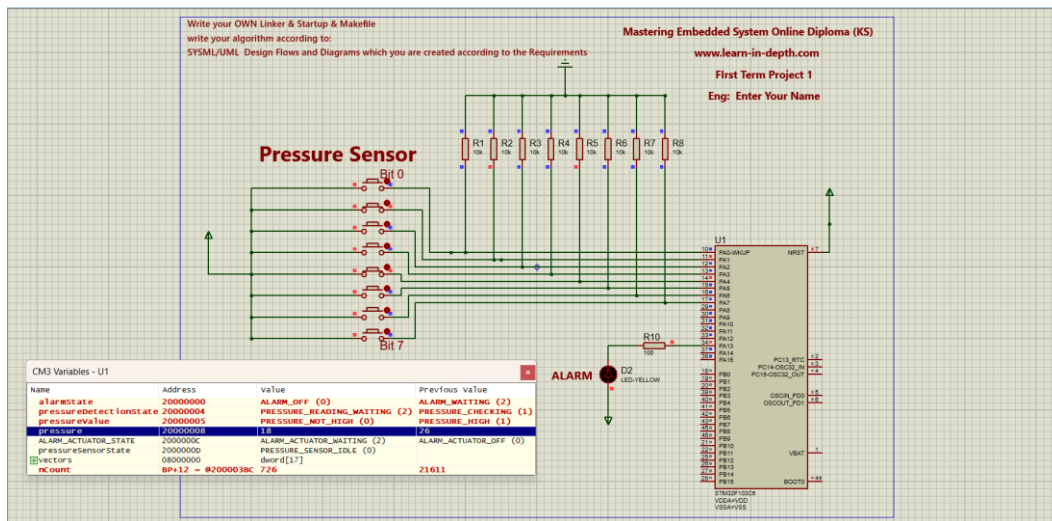


Figure 24 — Simulation — Alarm ON

When the pressure reading (26) exceeds the threshold (20):

- pressureValue = PRESSURE_HIGH (1)
- alarmState = ALARM_WAITING (2)
- ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_WAITING (2)
- Alarm LED (D2) is ON

→ Normal Pressure (pressure = 18, threshold = 20)



When the pressure reading (18) is below the threshold (20):

- pressureValue = PRESSURE_NOT_HIGH (0)
- alarmState = ALARM_OFF (0)
- ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_OFF (0)
- Alarm LED (D2) is OFF

Debug Watch Variables

VARIABLE	MODULE	DESCRIPTION
pressureSensorState	Pressure_Sensor_Driver.c	PRESSURE_SENSOR_IDLE (0) / PRESSURE_SENSOR_READING
pressureDetectionState	High_Pressure_Detection.c	PRESSURE_READING (0) / PRESSURE_CHECKING (1) / PRESSURE_READING_WAITING (2)
alarmState	Alarm.c	ALARM_OFF (0) / ALARM_ON / ALARM_WAITING (2)
ALARM_ACTUATOR_STATE	Alarm_Actuator_Driver.c	ALARM_ACTUATOR_OFF (0) / ALARM_ACTUATOR_ON / ALARM_ACTUATOR_WAITING (2)
pressure	High_Pressure_Detection.c	Current raw pressure reading
pressureValue	High_Pressure_Detection.c	PRESSURE_NOT_HIGH (0) / PRESSURE_HIGH (1)

10. Repository Structure

```

└─ Project_1_Pressure_Detection
    └─ makefile
    └─ toolchain.mk
    └─ src
        └─ main.c
        └─ High_Pressure_Detection.c
        └─ Alarm.c
        └─ driver
            └─ driver.c
            └─ Pressure_Sensor_Driver.c
            └─ Alarm_Actuator_Driver.c
    └─ inc
        └─ Platform_Types.h
        └─ High_Pressure_Detection.h
        └─ Alarm.h
        └─ driver
            └─ driver.h
            └─ Pressure_Sensor_Driver.h
            └─ Alarm_Actuator_Driver.h
    └─ startup
        └─ startup.c
    └─ linker
        └─ linkerscript.ld
    └─ Screen_Shots
        └─ UML/SysML diagrams
        └─ Code screenshots
        └─ Proteus simulation
    └─ build
```